INTELLIGENCE PROVIDERS

# HORIZON GRID

Provider Guide

**Version:** 0.2.5
**Documentation prepared:** 2026-08-23

PREPARED FOR EVALUATION

# Table of Contents

# Standalone Provider Guide

This guide is the complete, one-entry-per-provider reference for all 18 intelligence providers built into HORIZON GRID. *tech-04-provider-architecture.md* covers how the provider system works as a whole — the shared `BaseProvider` contract, the concurrent fan-out model, the Redis caching layer, and the runtime (no-restart) configuration architecture — and isn't repeated here at length. *user-06-providers.md* covers the same 18 providers from a plain-language, what-does-this-mean-for-an-analyst angle. This guide sits between them: for every provider, exactly what it is, what it needs, how to turn it on, how to prove it's working, and precisely how it fails when it fails.

## How to Read Every Entry Below

A few mechanics are identical across all 18 providers. Rather than repeat them 18 times, they're described once here; each provider's entry below only calls out what's actually different about that provider.

### How a result appears in an investigation

Every provider's answer — regardless of what the provider actually is — is rendered by the same frontend component (`ProviderCard.tsx`) into a uniform card: the provider's display name, a status badge (`ok`, `error`, `timeout`, `rate_limited`, `not_configured`, `unsupported_ioc`, `no_data`, or `disabled` — see the vocabulary below), a category badge, the real round-trip latency in milliseconds, a `cached` tag if the answer came from the Redis cache rather than a fresh outbound call, and a **Source** link to the specific page on the provider's own site backing that exact IOC (e.g. VirusTotal's own GUI search page for the value just looked up, not just VirusTotal's homepage).

Below that, every key in the connector's normalized `data` dictionary is rendered generically: short lists of simple values become small pill chips, single values become label/value rows, and anything more complex (nested objects, lists of objects) is deliberately left out of this friendly view and is only available by expanding the card's **Raw data** panel, which shows the exact JSON the connector produced. Once the AI has processed that specific provider's JSON (and only that provider's JSON — see *tech-03-ai-architecture.md*), a per-provider AI Summary appears underneath with a reputation line, a confidence level, and any interesting findings or relationships the model noticed.

The one provider whose findings don't fit the generic label/value layout — the Internet Intelligence Collector — gets its own dedicated rendering for the same reason: its `osint_findings` field is a list of attributed links (title, URL, snippet, source, published date), not scalar values, so the card renders it as an actual clickable finding list instead of collapsing it into the Raw data panel.

### Configuring a provider (Manage Providers UI)

Every provider is configured from the same place: the **Providers** page (`/providers`), **IOC Providers** tab. Each of the 18 providers gets its own row; expanding a row reveals exactly the credential field(s) that specific provider needs — no more, no fewer. A provider that needs no credential at all (6 of the 18, covered below) shows the plain statement "This provider needs no API key" in place of a form. Every credential field is a password-style input, and an already-saved credential is never shown in cleartext — the field's placeholder shows a masked preview (`****…` followed by the last few real characters) rather than the value itself. **Save** persists a typed credential via `POST /api/v1/runtime/ioc-providers/{provider_id}`; **Enable/Disable** flips the provider on or off at runtime via `POST /api/v1/runtime/ioc-providers/{provider_id}/enabled`. Both take effect starting with the very next investigation submitted anywhere on the platform — no restart, no redeploy.

### Testing a provider — and the retype-required behavior

Every provider row also has a **Test Connection** button. It calls `POST /api/v1/providers/{provider_id}/test`, which makes one real, minimal HTTP request against that provider's actual external API using the *candidate* credential value(s) currently typed into that row's form fields — never the already-saved, stored credential. This is deliberate: the test endpoint has no way to read the encrypted, stored credential (and no reason to — it exists specifically so an administrator can try out a not-yet-saved key without it ever touching the live configuration).

The practical consequence, worth stating plainly: **to re-test a credential you already saved and that's already working, you have to retype it into the field first.** If the field is left empty, "Test Connection" tests an empty string, and — correctly — reports that as a failure. That failure does not mean your saved, active key stopped working; it means the *test you just ran* had nothing to test. The field's masked placeholder is a visual hint that a value already exists there, not a stand-in value the test can silently use.

Where a provider has a real live test handler (12 of the 18 do — listed per-provider below), the response is one of several specific, genuine outcomes (authenticated success with real latency, a rejected/invalid key, a rate limit, or a network error) — never a generic pass/fail. Where a provider has no test handler at all (the same 6 providers that need no credential), clicking Test Connection still runs and still returns a real, honest response: the connector's fallback message, `"'{provider_id}' has no live connection test (it may require no key, e.g. Spamhaus, crt.sh, CISA KEV, MITRE ATT&CK, WHOIS/RDAP)."`

### Provider Health — is it actually working right now

Whether a provider is *configured* and whether it is *actually working* are two different questions, and the platform answers the second one on a dedicated **Provider Health** page (`/dashboard/provider-health`), built from real, database-backed aggregation of every call that provider has actually made — not a live check-in-the-moment. Each provider gets a status of Healthy, Degraded, Down, or Unknown, independently computed across four rolling windows (1 hour, 24 hours, 7 days, 30 days). Two guarantees hold for every one of the 18 providers without exception, and both were confirmed by real testing during this project, one of them after finding and fixing an actual bug in this exact calculation: a provider with **zero** real attempts in a window reports **Unknown**, never Healthy — silence is not evidence of health — and a provider that correctly answers "nothing found" (status `no_data` or `unsupported_ioc`) counts as a **healthy, successful outcome**, not a failure, because that is exactly what a working provider is supposed to say when its own dataset simply doesn't cover the indicator asked about.

### Who can do what — the standing RBAC rule for every provider

This is identical for all 18 providers, so it's stated once here rather than repeated 18 times below: **configuring, testing, enabling, or disabling any provider requires the `provider:manage` permission, which only the ADMIN role has** — ANALYST and VIEWER accounts cannot do any of it, including simply loading the Providers page's own data (`GET /api/v1/runtime/ioc-providers` is gated on `provider:manage` too, not just the write endpoints). Separately, **seeing a provider's results on an investigation requires only `lookup:read`, which all three roles hold** — ADMIN, ANALYST, and VIEWER can all view every provider card on every investigation, including one someone else ran. And **Provider Health is gated on `dashboard:read`**, which all three roles also hold, so any authenticated user can check whether a provider is actually working, even if only an ADMIN can change its credentials.

### The shared status vocabulary

Every provider result carries exactly one of these statuses (`app/providers/base.py`'s `ProviderStatus` enum), and none of them are ever confused for one another:

| Status | Meaning |
| --- | --- |
| `ok` | The provider was queried successfully and returned usable data (which may still say "clean"/"not listed"). |
| `no_data` | The provider was queried successfully but has nothing on this specific indicator — a genuine, honest empty result. |
| `unsupported_ioc` | This provider was never even called, because it doesn't support the IOC type submitted (e.g. a domain sent to Hybrid Analysis). |
| `not_configured` | This provider needs a credential and doesn't have one. |
| `disabled` | An administrator has deliberately turned this provider off at runtime — distinct from `not_configured`, since a provider can have a perfectly valid saved credential and still be switched off. |
| `error` | A genuine failure — a rejected credential, an unexpected response shape, or any other non-rate-limit fault. |
| `rate_limited` | The provider's own free-tier or quota limit was hit. |
| `timeout` | The provider didn't respond within the platform's timeout budget. |

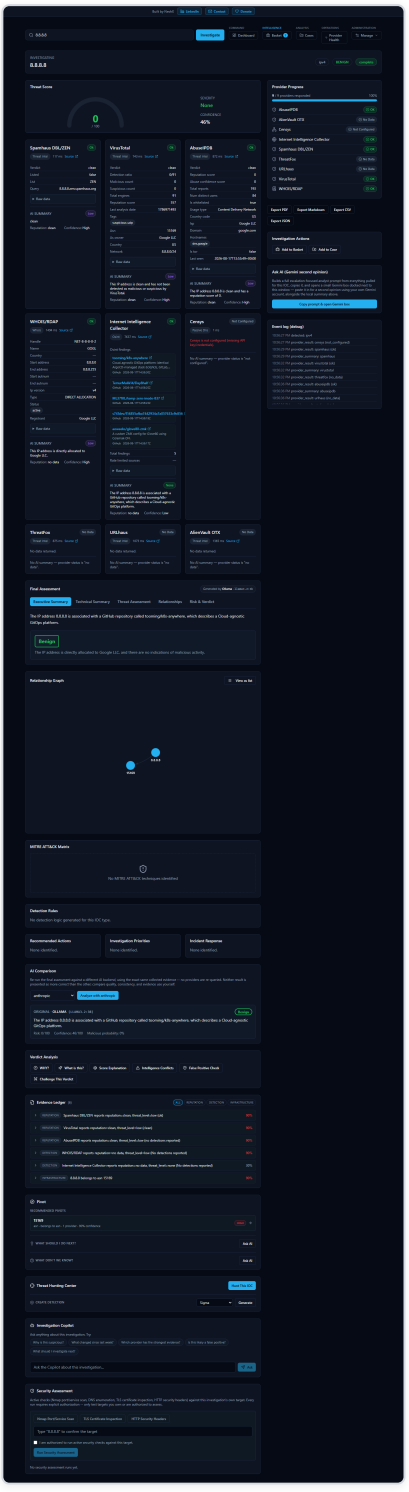

**Figure 1 —** Several provider cards on the same investigation showing different real statuses side by side — ok, not_configured, and rate_limited — each labeled distinctly rather than collapsed into a single generic "unavailable" state.

# Threat Intelligence Providers (10)

These are the reputation, blocklist, and community-intelligence sources — the platform's `threat_intel` category. Nine of the ten answer "has anyone seen this indicator before, and was it bad?"; the tenth, MITRE ATT&CK, is reference data rather than a reputation check and is called out as different where it matters.

## VirusTotal

- **Website:** https://www.virustotal.com
- **Category:** `threat_intel`
- **Purpose:** A multi-engine scanning aggregator — for a submitted hash, domain, IP, or URL, it reports what dozens of independent antivirus engines and URL/domain reputation scanners collectively think.
- **IOC types supported:** domain, ipv4, ipv6, md5, sha1, sha256, sha512, url.
- **API key required:** Yes.
- **Configure:** Providers page → IOC Providers → VirusTotal, one field: **Api Key**.
- **Test Connection:** Sends `GET /api/v3/ip_addresses/8.8.8.8` with the candidate key in the `x-apikey` header; HTTP 200 = valid key, 401 = "Authentication failed — check your API key," 429 = rate-limited-but-possibly-valid.
- **Provider Health:** Standard four-window tracking as described above.
- **What it returns / how it appears:** `verdict` (malicious/suspicious/clean/unknown, derived from `last_analysis_stats`), `detection_ratio` (e.g. "3/72"), `malicious_count` / `suspicious_count` / `total_engines`, `reputation_score`, `tags`; for hashes: file type, known file names, related malware family labels; for domains: resolved IPs and registrar; for URLs: the final URL after redirects and page title.
- **Error & failure handling:** A 404 from VirusTotal (indicator never submitted/scanned before) is mapped to `no_data`, not an error. Anything else that isn't a clean 200 falls through to `BaseProvider.run()`'s shared handling (429/403 → `rate_limited`, other HTTP errors → `error`).
- **Rate limits:** Public/free API v3 tier — 4 requests/minute, 500/day, per the connector's own module docstring.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## AbuseIPDB

- **Website:** https://www.abuseipdb.com
- **Category:** `threat_intel`
- **Purpose:** A community-reported IP-abuse database — network operators and analysts report IPs that attacked them (brute force, spam, scanning), so this reflects real-world abuse reports rather than automated scoring alone.
- **IOC types supported:** ipv4, ipv6.
- **API key required:** Yes.
- **Configure:** Providers page → IOC Providers → AbuseIPDB, one field: **Api Key**.
- **Test Connection:** Sends `GET /api/v2/check` for `8.8.8.8` with the candidate key in the `Key` header; 200 = valid, 401/403 = "Authentication failed," 429 = rate-limited.
- **Provider Health:** Standard four-window tracking.

- **What it returns / how it appears:** `verdict` (malicious if `abuseConfidenceScore` > 25, otherwise clean or unknown if zero reports exist), `abuse_confidence_score`, `total_reports`, `num_distinct_users`, `is_whitelisted`, `usage_type`, `is_tor`, and the actual list of abuse `reports`.
- **Error & failure handling:** A 404 or an empty `data` object both map to `no_data` — a real, checked-and-clean answer, not an error. Genuine HTTP failures fall through to the shared 429/403 → `rate_limited`, other → `error` handling.
- **Rate limits:** v2 free tier — 1,000 checks/day, per the connector's own module docstring.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## AlienVault OTX

- **Website:** https://otx.alienvault.com
- **Category:** `threat_intel`
- **Purpose:** A large, community-contributed threat-intelligence exchange organized around "pulses" — analyst- and vendor-published write-ups linking indicators to named malware families, threat actors, and campaigns. Useful for context, not just a bare verdict.
- **IOC types supported:** domain, hostname, ipv4, ipv6, md5, sha1, sha256, sha512, url.
- **API key required:** Yes.
- **Configure:** Providers page → IOC Providers → AlienVault OTX, one field: **Api Key**.
- **Test Connection:** Sends `GET /api/v1/indicators/IPv4/8.8.8.8/general` with the candidate key in the `X-OTX-API-KEY` header; 200 = valid, 401/403 = "Authentication failed."
- **Provider Health:** Standard four-window tracking — worth calling out specifically here: OTX legitimately returns `no_data` (zero matching pulses) for the majority of real-world lookups, since most indicators simply aren't referenced in any published pulse. This is exactly the scenario a real pre-release bug in the health calculation once miscounted as a failure — OTX was measured at a misleading 64% "Degraded" under the old (incorrect) logic and correctly measured at 100% "Healthy" once fixed. See *user-12-health-troubleshooting.md* for the full account.
- **What it returns / how it appears:** `verdict` (malicious if `pulse_count` > 0, else unknown), `pulse_count`, `malware_families`, `threat_actors`, `campaigns`, `tags`, up to 25 `references`, plus type-specific fields (ASN/country for IPs, WHOIS/Alexa rank for domains, resolved domain for URLs).
- **Error & failure handling:** A 404, or a 200 with zero pulses, both map to `no_data`. Other HTTP failures use the shared 401/403 → `error`, 429 → `rate_limited` mapping.
- **Rate limits:** Not numerically documented in the connector; governed by AlienVault's own account tier, surfaced back through the shared HTTP-status mapping if hit.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## URLhaus

- **Website:** https://urlhaus.abuse.ch
- **Category:** `threat_intel`
- **Purpose:** An abuse.ch project tracking URLs actively distributing malware — catches live malware-hosting infrastructure specifically, not general reputation.
- **IOC types supported:** url, domain, ipv4 (domain/IP are queried as URLhaus "host" lookups).
- **API key required:** Yes — a free abuse.ch account Auth-Key, shared with ThreatFox and MalwareBazaar (one key activates all three).
- **Configure:** Providers page → IOC Providers → URLhaus, one field: **Auth Key**.

- **Test Connection:** Uses the shared abuse.ch handler — sends a ThreatFox `search_ioc` query (the lightest of the three abuse.ch endpoints) with the candidate key in the `Auth-Key` header, and reads the JSON `query_status` field rather than the HTTP status code, since abuse.ch's APIs return HTTP 200 even on a rejected key.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** For a URL lookup — `verdict: "malicious"`, `url_status`, `threat`, `tags`, `related_hashes` (payload SHA256s), first/last-seen dates, `blacklists`. For a host (domain/IP) lookup — `related_urls`, `url_count`, aggregated `tags` / `threat` across every URL hosted there.
- **Error & failure handling:** This is a deliberately important case. abuse.ch's `query_status` vocabulary distinguishes a genuine empty result (`"no_results"`, `"hash_not_found"`) from an authentication failure (`"no_api_key"`, `"invalid_api_key"`, `"unauthorized"`) — and the shared `app/providers/abusech.py` module maps the auth-failure case to `ProviderStatus.ERROR`, **never** `no_data`. The reason is explicit in the module's own docstring: a misconfigured key returning `no_data` would look, on screen, exactly like "checked and found nothing" for a possibly-malicious IOC — mapping it to `error` instead makes a bad key impossible to mistake for a clean result.
- **Rate limits:** Not numerically documented in the connector; a genuine HTTP-level 429 from abuse.ch (distinct from the soft, 200-status auth failure above) is still caught by the shared retry/status handling and reported as `rate_limited`.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## ThreatFox

- **Website:** https://threatfox.abuse.ch
- **Category:** `threat_intel`
- **Purpose:** A second abuse.ch project — a shared, community-contributed IOC database covering a broader set of indicator types than URLhaus's URL-distribution focus.
- **IOC types supported:** domain, ipv4, ipv6, md5, sha256, url.
- **API key required:** Yes — the same shared abuse.ch Auth-Key as URLhaus and MalwareBazaar.
- **Configure:** Providers page → IOC Providers → ThreatFox, one field: **Auth Key**.
- **Test Connection:** Same shared abuse.ch handler described under URLhaus.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict: "malicious"` (ThreatFox only returns matches, never a "clean" verdict), `threat_type`, `malware_families`, `confidence_level` (abuse.ch's own submitted confidence), first/last-seen dates, `tags`, a reference link, and the full raw `matches` list.
- **Error & failure handling:** Same abuse.ch `query_status` handling as URLhaus — `"ok"` with zero entries is treated as a genuine `no_data` result (explicitly distinguished in the code from an auth failure, which still maps to `error`).
- **Rate limits:** Same as URLhaus — not numerically documented; shared Auth-Key, shared abuse.ch-side limits.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## MalwareBazaar

- **Website:** https://bazaar.abuse.ch
- **Category:** `threat_intel`
- **Purpose:** The third abuse.ch project — a repository of actual submitted malware sample hashes. A match means someone has already submitted that exact malicious file to the community.
- **IOC types supported:** md5, sha1, sha256, sha512.
- **API key required:** Yes — the same shared abuse.ch Auth-Key as URLhaus and ThreatFox.

- **Configure:** Providers page → IOC Providers → MalwareBazaar, one field: **Auth Key**.
- **Test Connection:** Same shared abuse.ch handler described under URLhaus.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict: "malicious"`, `file_type` / `file_type_mime` / `file_size` / `file_name`, `malware_families` (from the sample's `signature`), `delivery_method`, `tags`, first/last-seen dates, and (when present) `vendor_intel` — third-party sandbox commentary abuse.ch itself has aggregated for that sample.
- **Error & failure handling:** Same abuse.ch `query_status` handling — auth failure → `error`, genuine empty result (`"hash_not_found"`, `"illegal_hash"`) → `no_data`.
- **Rate limits:** Same as URLhaus — not numerically documented; shared Auth-Key.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## Google Safe Browsing

- **Website:** https://safebrowsing.google.com (API: https://developers.google.com/safe-browsing/v4/lookup-api)
- **Category:** `threat_intel`
- **Purpose:** Google's own phishing/malware/unwanted-software URL and domain reputation check — the same list that flags sites in Chrome. See the dedicated deep-dive section below for this provider's full safety-guarantee treatment.
- **IOC types supported:** url, domain (a bare domain is submitted with an `http://` scheme added, since the API's `threatEntries.url` field expects a full URL).
- **API key required:** Yes.
- **Configure:** Providers page → IOC Providers → Google Safe Browsing, one field: **Api Key**.
- **Test Connection:** Sends the exact same `threatMatches:find` request the real connector uses, against `http://google.com/`, with the candidate key as a query parameter; 200 = valid, 400/401/403 = "rejected the request — check your API key," 429 = quota exceeded.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict` (`clean`, `malicious`, or `unknown` — see the deep dive below for exactly which inputs are allowed to produce each), `matches` (the raw threat-match entries when malicious), `threat_types` (e.g. `MALWARE`, `SOCIAL_ENGINEERING`), `match_count`.
- **Error & failure handling:** See the deep dive below — this is one of the two providers built with an explicit, tested guarantee that a failure can never look like a clean/safe result.
- **Rate limits:** Not numerically documented in the connector; Google's own quota is enforced server-side and surfaces as HTTP 429, mapped explicitly to `rate_limited`.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## Spamhaus DBL/ZEN

- **Website:** https://www.spamhaus.org
- **Category:** `threat_intel`
- **Purpose:** A long-established DNS-based blocklist (DNSBL) covering IPs (ZEN, the combined SBL+XBL+PBL list) and domains (DBL) — widely used across email and network security. Unlike every other provider in this platform, it's checked via a raw DNS query, not an HTTP API call.
- **IOC types supported:** ipv4, domain.
- **API key required:** No.

- **Configure:** Providers page → IOC Providers → Spamhaus DBL/ZEN — shows "This provider needs no API key," with only an Enable/Disable toggle.
- **Test Connection:** No live handler exists (nothing to authenticate). Clicking it returns the connector's generic fallback message explaining no key is required.
- **Provider Health:** Standard four-window tracking — Spamhaus legitimately returns "not listed" (a healthy `ok` result, not `no_data`) for the overwhelming majority of lookups, since most IPs/domains simply aren't on either list.
- **What it returns / how it appears:** A DNS A-record answer in the 127.0.0.0/8 range signals a listing; no answer (NXDOMAIN) means not listed. Returns `verdict` (`clean` / `malicious`), `listed` (boolean), `list` (`"ZEN"` or `"DBL"`), the literal `query` name that was resolved, and — if listed — the raw `return_codes` plus a human-readable `listing_reason` decoded from Spamhaus's own published return-code table (e.g. `"XBL - CBL: compromised host (open proxy / worm / bot infection)"`).
- **Error & failure handling:** A DNS resolution failure (`socket.gaierror`, i.e. NXDOMAIN) is the expected "not listed" answer, mapped to a clean `ok` result rather than an error — there is no separate "down" state for this provider's actual check, since a failed DNS lookup for this specific query pattern *is* the documented not-listed signal.
- **Rate limits:** No HTTP rate limit (it's DNS), but Spamhaus's own published return-code tables include dedicated codes for exactly this situation, hardcoded into the connector's lookup tables: `127.255.255.255` ("query error – excessive number of queries, temporarily blocked") and `127.255.255.254` ("query error – public/open resolver not permitted to query Spamhaus") for ZEN, and `127.0.1.255` ("query error – IP queries prohibited, misconfigured resolver") for DBL. If ever triggered, these surface as a "listed" result whose `listing_reason` names the query-error code rather than a real detection.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## PhishTank

- **Website:** https://www.phishtank.com
- **Category:** `threat_intel`
- **Purpose:** A community-reported and community-verified phishing URL database. Since phishing pages are typically short-lived, a frequently updated, dedicated phishing feed catches things a slower-moving general reputation feed may not yet have.
- **IOC types supported:** url.
- **API key required:** No — PhishTank's `checkurl` API works unauthenticated; an optional key (`app_key`) only raises the rate limit if supplied.
- **Configure:** Providers page → IOC Providers → PhishTank — shows "This provider needs no API key" for the required flow, though an optional key can still be entered via the wizard/`.env` path for a higher limit.
- **Test Connection:** Sends the real `checkurl` request against a benign test URL. Note a real, documented quirk in the test handler itself: PhishTank can return HTTP 403 to *any* generic HTTP client — including the platform's own production connector, using this exact request shape — independent of whether a key is supplied or valid. The test treats a 403 here as inconclusive-but-OK rather than a hard failure, explaining that no key is actually required and the platform will still query PhishTank normally during real investigations.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict` (`malicious` if PhishTank's own `valid` flag is true, `suspicious` if reported-but-unconfirmed), `in_database`, `verified` / `verified_at`, `submission_time`, `phish_id`, and a direct link to PhishTank's own detail page for that report.
- **Error & failure handling:** A 404, or a response where `in_database` is false, both map to `no_data` — genuinely "not in the database," not an error.
- **Rate limits:** HTTP 509 is PhishTank's own documented over-limit response code (per a code comment in `app/providers/base.py`), explicitly mapped to `rate_limited` rather than falling into the generic `error` bucket other unexpected statuses get.

- **Roles:** Configure — ADMIN only ( `provider:manage` ). View results — any role with `lookup:read` .

## MITRE ATT&CK

- **Website:** https://attack.mitre.org
- **Category:** `threat_intel` (by registry classification — functionally, this is reference data, not a reputation check, and is the one provider in this section that doesn't answer "is this bad")
- **Purpose:** Looks up an official ATT&CK technique ID (e.g. `T1059` or its sub-technique `T1059.001` ) against the real, public MITRE CTI STIX bundle for the Enterprise ATT&CK matrix, returning its real name, description, and tactics. It exists mainly to enrich MITRE technique references the correlation engine has already identified elsewhere in an investigation with the official name and description, rather than leaving a bare technique ID on screen.
- **IOC types supported:** mitre_technique.
- **API key required:** No — this is public reference data.
- **Configure:** Providers page → IOC Providers → MITRE ATT&CK — shows "This provider needs no API key," Enable/Disable only.
- **Test Connection:** No live handler exists; returns the generic fallback message.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `technique_id` , `name` , `description` , `tactics` (kill-chain phase names), `x_mitre_platforms` , `x_mitre_data_sources` , `is_subtechnique` , and whether the technique has been `revoked` or `deprecated` in the current ATT&CK version.
- **Error & failure handling:** A technique ID not found in the current bundle maps to `no_data` .
- **Rate limits:** None applicable — this queries a static, public GitHub-hosted STIX bundle (~30 MB), which the connector keeps in an in-memory cache refreshed at most once per hour, guarded by an `asyncio.Lock` so a cold/expired cache under concurrent lookups triggers exactly one refetch, not one per concurrent request.
- **Roles:** Configure — ADMIN only ( `provider:manage` ). View results — any role with `lookup:read` .

---

# Sandbox Providers (2)

These execute or scan the target live, rather than only consulting a pre-built database.

## urlscan.io

- **Website:** https://urlscan.io
- **Category:** `sandbox`
- **Purpose:** A live URL/domain sandbox scanner — submits the target for an actual scan (page render, screenshot, network/DOM capture) and reports the result, rather than only consulting a static reputation list. See the dedicated deep-dive section below for this provider's full safety-guarantee treatment.
- **IOC types supported:** url, domain (a bare domain is submitted as `http://<domain>`, since urlscan's scan endpoint requires a full URL).
- **API key required:** Yes.
- **Configure:** Providers page → IOC Providers → urlscan.io, one field: **Api Key**.
- **Test Connection:** Sends the real submission request (`POST /api/v1/scan/`, `visibility: "unlisted"`) against a benign test domain, using the candidate key in the `API-Key` header — checks only the submission response, not full scan completion, since this is a credential check, not a real investigation. 200/201 = valid, 401/403 = "rejected the API key," 429 = rate limit exceeded.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict` (`malicious` / `clean` / `unknown`, taken directly from urlscan's own boolean verdict field — see the deep dive below for why it never infers this from a raw score), `score`, `categories`, `brands`, the scan's `final_url` / `domain` / `ip` / `country` / `asn`, every `resolved_ips` and `domains_contacted` the page actually reached, and a direct `screenshot_url` for the scan.
- **Error & failure handling:** See the deep dive below.
- **Rate limits:** Not numerically documented for submission itself; the connector enforces its own 60-second wall-clock budget for the whole submit-then-poll cycle (`_TIMEOUT_SECONDS`), polling for the finished result every 3 seconds (`_POLL_INTERVAL_SECONDS`) — this is the platform's own choice to bound how long one investigation can wait on one provider, not a urlscan.io-imposed limit. A real HTTP 429 from urlscan.io itself, on either the submit or poll call, is mapped explicitly to `rate_limited`.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

## Hybrid Analysis (Falcon Sandbox)

- **Website:** https://hybrid-analysis.com
- **Category:** `sandbox`
- **Purpose:** CrowdStrike's Falcon malware sandbox — files are actually executed in an isolated environment and their real behavior recorded. This platform checks whether a prior sandbox run already exists for the exact file hash submitted.
- **IOC types supported:** sha256 only. This is a deliberate, documented restriction, not an oversight: the connector's own module docstring notes that Hybrid Analysis's older hash-search endpoint (which historically also accepted MD5/SHA1) is confirmed live to return HTTP 410 Gone, and the working replacement endpoint (`/api/v2/overview/{sha256}/summary`) only ever accepts a SHA256. MD5/SHA1 support was removed rather than left silently broken.
- **API key required:** Yes.

- **Configure:** Providers page → IOC Providers → Hybrid Analysis (Falcon Sandbox), one field: **Api Key**.
- **Test Connection:** Sends `GET /api/v2/overview/{sha256}/summary` for the SHA256 of an empty string (a real, safe, universally-known test value) with the candidate key in the `api-key` header; 200 or 404 both count as a valid key (404 just means that exact hash hasn't been scanned), 401/403 = "Authentication failed."
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict` (the sandbox's own verdict string, e.g. `malicious` / `no specific threat`), `threat_score`, `av_detect_pct` (multi-scan result), and first/last-seen submission dates.
- **Error & failure handling:** A 400 or 404 both map to `no_data` — no prior sandbox run exists for that exact hash, which is the common, expected case rather than a fault.
- **Rate limits:** Not numerically documented in the connector; falls through to the shared 429/403/509 → `rate_limited`, other HTTP errors → `error` mapping.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

---

# Certificate Intelligence Providers (1)

## crt.sh

- **Website:** https://crt.sh
- **Category:** `certificate_intel`
- **Purpose:** Searches the public, tamper-evident Certificate Transparency logs that every publicly trusted TLS certificate must be logged in. This is a well-known technique for discovering other subdomains and infrastructure tied to the same certificate — useful for mapping an attacker's broader footprint from just one domain.
- **IOC types supported:** domain, tls_certificate.
- **API key required:** No — crt.sh is a free community service.
- **Configure:** Providers page → IOC Providers → crt.sh — shows "This provider needs no API key," Enable/Disable only.
- **Test Connection:** No live handler exists; returns the generic fallback message.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** Up to the 25 most recent matching `certificates` (issuer, common name, all names covered by the cert, validity dates, serial number), a total `certificate_count`, and a deduplicated list of `related_domains` discovered across every certificate's Subject Alternative Names.
- **Error & failure handling:** crt.sh is a free, community-run Postgres-backed service that the connector's own docstring describes as "sometimes slow/flaky," occasionally returning malformed or truncated JSON under load. Rather than let that raise an uncaught error, a JSON parse failure is explicitly caught and degraded to `no_data` with an `error_message` noting the malformed response — a real reliability characteristic of the upstream service, handled deliberately rather than left to crash the lookup.
- **Rate limits:** None documented; the "slow/flaky under load" behavior above is a reliability characteristic of the free service, not a hard rate limit.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

---

# Vulnerability Providers (2)

These answer a different kind of question than the rest of this guide: not "is this indicator bad," but "how serious is this known software vulnerability, and is it actually being exploited."

## NIST NVD

- **Website:** https://nvd.nist.gov
- **Category:** `vulnerability`
- **Purpose:** The U.S. government's official, authoritative catalog of publicly disclosed software vulnerabilities. Given a CVE ID, returns the full description and CVSS severity score — usually an analyst's starting point for understanding what a vulnerability actually is.
- **IOC types supported:** cve.
- **API key required:** No — works unauthenticated at a lower rate limit; an optional key raises the limit.
- **Configure:** Providers page → IOC Providers → NIST NVD, one optional field: **Api Key**.
- **Test Connection:** Sends `GET /rest/json/cves/2.0` for `CVE-2021-44228` (Log4Shell, a real published CVE) with the candidate key (if any) in the `apiKey` header; 200 = valid, 403 = "Authentication failed," 429 = rate-limited.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict` derived from CVSS severity ( `CRITICAL` / `HIGH` → malicious, `MEDIUM` / `LOW` → suspicious), `cvss_score` , `cvss_severity` , `cvss_vector` (preferring CVSS v3.1, falling back to v3.0 then v2), full English `description` , `vuln_status` , associated `cwes` (weakness categories), and `references` .
- **Error & failure handling:** A 404 or a response with zero `vulnerabilities` both map to `no_data` .
- **Rate limits:** ~5 requests/30 seconds unauthenticated; ~50 requests/30 seconds with an API key — both per the connector's own module docstring.
- **Roles:** Configure — ADMIN only ( `provider:manage` ). View results — any role with `lookup:read` .

## CISA Known Exploited Vulnerabilities (KEV)

- **Website:** https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- **Category:** `vulnerability`
- **Purpose:** Also a U.S. government source, but answering a sharper question than NVD: is this vulnerability *confirmed* to be actively exploited by real attackers right now, not just theoretically dangerous? A CVE's presence in this catalog is exactly the kind of finding that should jump it to the top of a patching queue.
- **IOC types supported:** cve.
- **API key required:** No.
- **Configure:** Providers page → IOC Providers → CISA Known Exploited Vulnerabilities — shows "This provider needs no API key," Enable/Disable only.
- **Test Connection:** No live handler exists; returns the generic fallback message.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `verdict: "malicious"` (presence in this catalog is itself the finding), `vulnerability_name` , `vendor_project` , `product` , `date_added` , `short_description` , CISA's own `required_action` and `due_date` , and whether it's flagged for `known_ransomware_campaign_use` .

- **Error & failure handling:** A CVE not present in the catalog maps to `no_data` — most CVEs are not actively exploited, so this is the common, expected result.
- **Rate limits:** None applicable — the connector fetches CISA's full ~1 MB JSON catalog once and keeps an in-memory cache refreshed at most once per hour, guarded by an `asyncio.Lock` (the same pattern MITRE ATT&CK uses) so a cold cache under concurrent lookups triggers exactly one refetch, not one per request.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

---

# WHOIS Providers (1)

## WHOIS/RDAP

- **Website:** https://www.whois.com (general WHOIS reference); the connector itself talks directly to each TLD's own port-43 WHOIS server for domains, and to https://rdap.org — a bootstrap service that automatically routes to whichever regional registry (ARIN/RIPE/APNIC/LACNIC/AFRINIC) actually holds the record — for IPs and ASNs.
- **Category:** `whois`
- **Purpose:** Answers "who owns this, and when was it registered" — often the very first thing an analyst checks: is this a domain registered yesterday, or a major cloud provider's address range that's been allocated for years?
- **IOC types supported:** domain, ipv4, ipv6, asn.
- **API key required:** No.
- **Configure:** Providers page → IOC Providers → WHOIS/RDAP — shows "This provider needs no API key," Enable/Disable only.
- **Test Connection:** No live handler exists; returns the generic fallback message.
- **Provider Health:** Standard four-window tracking — like OTX and Spamhaus, a large share of real lookups legitimately return `no_data` (a domain a TLD's WHOIS server has nothing usable to say about), and that correctly counts as healthy, not degraded.
- **What it returns / how it appears:** For domains — `registrar`, `creation_date` / `expiration_date` / `updated_date`, `name_servers`, `registrant` / `registrant_country`, `status`. For IPs/ASNs (via RDAP) — `handle`, block `name`, `country`, the address range (`start_address` / `end_address` or `start_autnum` / `end_autnum`), `entities` (registrant/admin/tech contacts), and, where the RIR's RDAP extension exposes it, originating `asn` values for that IP block.
- **Error & failure handling:** This connector distinguishes three genuinely different situations rather than collapsing them into one. A domain with no usable WHOIS record, or a response `python-whois` can't parse (many TLDs return non-standard text), maps to `no_data`. A real, live infrastructure failure — the TLD's port-43 WHOIS server refusing the connection, timing out, or resolving to nothing — maps to `timeout` or `error` instead, specifically because that's a different, and more actionable, situation than "this domain simply has no WHOIS record." The connector's own code comments document a real, deliberate fix here: the underlying `python-whois` library's default `ignore_socket_errors=True` behavior used to swallow a genuine socket failure into response *text* instead of raising, which then silently fell into the same "no data" branch as an actually-unregistered domain — that default is explicitly overridden so a real socket failure raises and gets reported honestly as `timeout` / `error`.
- **Rate limits:** None documented; a 10-second socket timeout is enforced per WHOIS query so one slow TLD server can't hang the request indefinitely.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

---

# Passive DNS Providers (1)

## Censys

- **Website:** https://censys.io (queried via the newer Censys Platform API at platform.censys.io)
- **Category:** `passive_dns` (by registry classification — functionally, this is internet-wide host/service scan data: what's actually running on a given IP right now, rather than a DNS resolution history)
- **Purpose:** An internet-wide scanning service cataloging what's actually running on IP addresses across the internet — open ports, running services, hosting location, and the autonomous system it belongs to. Useful for understanding what an IP is actually hosting, beyond a bare reputation score.
- **IOC types supported:** ipv4, ipv6.
- **API key required:** Yes — and Censys is the one provider in the platform that needs **two** separate credentials, both required: a Personal Access Token *and* the Organization ID that token belongs to. Having only one leaves it `not_configured`.
- **Configure:** Providers page → IOC Providers → Censys, two fields: **Personal Access Token** and **Organization Id**.
- **Test Connection:** Sends `GET /v3/global/asset/host/8.8.8.8` with the candidate token as a Bearer `Authorization` header and the candidate org ID in `X-Organization-ID`; 200 or 404 both count as valid credentials, 401/403 = "Authentication failed — check your access token and organization ID." If the organization ID is left blank, the test fails immediately with "Organization ID is required in addition to the access token," without even making the HTTP call.
- **Provider Health:** Standard four-window tracking.
- **What it returns / how it appears:** `services` (port, transport protocol, service name for everything Censys has observed open on that host), `location` (city/province/country), `autonomous_system` (and a top-level `asn` / `as_owner` convenience field), and `last_seen` (when Censys last updated its view of that host).
- **Error & failure handling:** A 404 (host not in Censys's dataset) maps to `no_data`. Missing either credential is caught before any HTTP call is even made, reported as `not_configured` rather than a network error.
- **Rate limits:** Not numerically documented in the connector; falls through to the shared 429/403 → `rate_limited` mapping.
- **Roles:** Configure — ADMIN only (`provider:manage`). View results — any role with `lookup:read`.

---

# OSINT Providers (1)

## Internet Intelligence Collector

- **Website:** Not a single external site — this is the platform's own live crawler, wrapping four independent open-source-intelligence sources: GitHub, Reddit, RSS security-news feeds, and best-effort paste-dump search ( `app/crawler/sources/github.py` , `reddit.py` , `rss_news.py` , `pastebin_search.py` ).
- **Category:** `osint`
- **Purpose:** Unlike every other provider in this platform, which queries a fixed, pre-built database, this one goes out and fetches results live, at the moment the investigation runs. That makes it uniquely able to surface very recent, informal chatter — a proof-of-concept exploit just published on GitHub, a fresh Reddit thread — that a slower-moving commercial feed hasn't caught up to yet. Every finding keeps its original source URL, so nothing is ever presented without attribution.
- **IOC types supported:** domain, ipv4, campaign, cve, file_name, malware_family, threat_actor. (Raw network atoms like a JA3 hash or a mutex string are deliberately excluded — free-text search on them is almost always noise, not signal.)
- **API key required:** No — every underlying source is queried unauthenticated / best-effort.
- **Configure:** Providers page → IOC Providers → Internet Intelligence Collector — shows "This provider needs no API key," Enable/Disable only.
- **Test Connection:** No live handler exists; returns the generic fallback message.
- **Provider Health:** Standard four-window tracking — like OTX, Spamhaus, and WHOIS/RDAP, a `no_data` result (no relevant public chatter found for this specific indicator, which is the common case for most indicators) is a correct, healthy outcome, not a failure.
- **What it returns / how it appears:** `osint_findings` — a deduplicated (by URL), capped list of `{title, url, snippet, published_at, source}` records, rendered in the investigation as an actual clickable, attributed finding list (see "How a result appears," above) rather than the generic label/value layout every other provider uses. Also reports a per-source `source_count` breakdown and any `rate_limited_sources` .
- **Error & failure handling:** Each of the four underlying sources is queried concurrently and its failure is isolated — one source failing or being rate-limited never blocks the other three from contributing results. A specific, deliberate distinction is made in the aggregate status: if every source that came back empty was empty because it was rate-limited (not because there's genuinely nothing to find), the overall result is reported as `rate_limited` , not `no_data` — specifically so a rate-limited crawl is never mistaken for "nothing found," which could otherwise look like a clean, checked-and-clear indicator.
- **Rate limits:** Each of the four source modules can independently raise a rate-limit condition ( `SourceRateLimitedError` ), tracked per-source; the collector itself caps the total merged result set at `settings.crawler_max_results_per_source` × 4 sources.
- **Roles:** Configure — ADMIN only ( `provider:manage` ). View results — any role with `lookup:read` .

---

# Deep Dive: The "UNKNOWN Is Not SAFE" Guarantee

urlscan.io and Google Safe Browsing get separate, deeper treatment here because of what a wrong answer from either one would actually mean in practice. Both are used, in effect, to answer "is this specific URL/domain safe to open right now?" — and for both, the single most damaging kind of bug imaginable is not a crash or a slow response; it's a *false negative*: a genuine API failure (an invalid key, an outage, a rate limit) getting silently rendered on screen as "clean" or "safe." An analyst who trusts that a URL scanned "clean" when the provider that was supposed to check it actually never got a real answer is worse off than an analyst who was told plainly "this check didn't run."

Both connectors were built, from the start, around a single non-negotiable invariant: **a genuine failure is always reported as `error`, `timeout`, or `rate_limited` — never anything that could be rendered on screen as a clean or safe result.**

## Google Safe Browsing's invariant, stated directly in its own code

`app/providers/google_safe_browsing.py`'s module docstring states the rule in exactly these words:

> *"CRITICAL invariant, enforced throughout this module: an empty/missing `matches` field on a genuine HTTP 200 response is the ONLY input that may produce a 'clean' verdict. Every other outcome — a non-200 status, a network error/timeout, or a response body that doesn't parse the way the API contract promises — returns early via `_error()`, which always sets `data={"verdict": "unknown", ...}` on the ProviderResult it builds. There is no code path from 'the request failed' to a result whose `data` looks like a clean scan."*

Concretely, every one of these situations — a timeout, a network error, HTTP 400/401/403 (bad key), HTTP 429 (quota exceeded), any other non-200 status, a response body that fails to parse as JSON, a non-object response, or a `matches` field that isn't the shape the API contract promises — routes through the exact same `_error()` helper, which unconditionally sets `data={"verdict": "unknown", "matches": [], "threat_types": []}` and never anything that resembles a real clean scan. The *only* code path that can ever produce `verdict: "clean"` is a confirmed HTTP 200 response whose `matches` field is genuinely absent or an empty list — i.e., Google's own API telling this connector, unambiguously, "checked, nothing found."

This isn't just a design claim — it's pinned down by a dedicated, named set of unit tests (`app/tests/unit/test_google_safe_browsing.py`) whose test names spell out exactly what they're proving: `test_401_maps_to_error_never_looks_like_safe`, `test_403_maps_to_error_never_looks_like_safe`, `test_400_maps_to_error_never_looks_like_safe`, `test_429_maps_to_rate_limited_never_looks_like_safe`, `test_500_maps_to_error_never_looks_like_safe`, `test_malformed_json_maps_to_error_never_looks_like_safe`, `test_non_object_response_maps_to_error_never_looks_like_safe`, `test_malformed_matches_field_maps_to_error_never_looks_like_safe`, `test_timeout_maps_to_timeout_never_looks_like_safe`, and `test_network_error_maps_to_error_never_looks_like_safe`.

## urlscan.io's invariant: never fabricate a verdict, and never fabricate a result from an incomplete scan

urlscan.io's connector (`app/providers/urlscan_io.py`) enforces the same underlying principle through a submit-then-poll model, with two distinct failure modes guarded against:

1. **A genuine API/credential failure never becomes a verdict.** The submission call (`POST /api/v1/scan/`) and every poll call (`GET /api/v1/result/{uuid}/`) each check their HTTP status explicitly, before the shared `raise_for_status()`-based generic

handling would even run — because for this specific provider, HTTP 401/403 mean "bad API key," not "rate limited," which is different from what the platform's shared status mapping assumes by default for most other connectors (which treat 403 as a rate-limit signal, matching PhishTank's documented behavior). Getting this distinction right required deliberately *not* relying on the generic mapping — the connector's own module docstring calls this out explicitly as a departure from the shared default, made on purpose.

2. **An incomplete scan never becomes a fabricated result.** A scan that isn't finished yet returns HTTP 404 from urlscan.io's own result endpoint — the documented "not ready" signal — and the connector polls again rather than treating that as "no data." But polling can't continue forever: a hard 60-second wall-clock budget ( `_TIMEOUT_SECONDS` ) bounds the whole submit-then-poll cycle, and if the scan simply hasn't finished by then, the connector reports `timeout` — it does **not** return a "clean" or partial result assembled from whatever incomplete data might exist at that point.

When a result does come back complete, the connector also refuses to *guess* a verdict from a raw numeric score: it trusts only urlscan.io's own explicit boolean `malicious` verdict field (checking the `overall` verdict object first, falling back to the `urlscan` engine's own sub-object), and reports `unknown` — not a guessed `clean` — whenever neither field is present. The code comment is direct about why: the score scale and direction differ across urlscan's various verdict sub-objects, and guessing a threshold "would risk fabricating a verdict the API never actually asserted."

This, too, is pinned by dedicated tests in `app/tests/unit/test_urlscan_io.py` : `test_submission_401_maps_to_error_not_rate_limited` , `test_submission_403_maps_to_error_not_rate_limited` , `test_submission_429_maps_to_rate_limited` , `test_poll_403_maps_to_error_not_rate_limited` , `test_poll_429_maps_to_rate_limited` , `test_scan_never_ready_maps_to_timeout_not_a_fabricated_result` , and `test_absent_malicious_signal_maps_to_unknown_not_fabricated_clean` .

## The proof: a real, live chaos test with deliberately invalid credentials

Unit tests prove the code's logic; they don't prove the real, live external APIs actually behave the way the code assumes they will. As part of this project's final release QA pass, both providers were tested live, against their real production APIs, using **deliberately invalid API keys** — not mocked responses. The result, as recorded in the project's own release QA report ( `FINAL_RELEASE_QA_REPORT.md` ):

> *"A provider failure never reads as 'safe' — Live test: invalid API keys → `status=error` , Safe Browsing `verdict=unknown` , final_verdict=UNKNOWN (never benign/clean) — **PASS**."*

Concretely: the invalid key was rejected by the real external API with `status=error` on the provider result, Google Safe Browsing's own `verdict` field read `"unknown"` — never `"clean"` — and, critically, the investigation's own overall final verdict (the thing an analyst actually reads first) was reported as `UNKNOWN` rather than defaulting to anything that could be read as benign or safe. This live test is what elevated the guarantee from "the code appears to do the right thing" to "confirmed, against the real vendor APIs, that it actually does." The release QA report's feature matrix (27 unit tests across both providers, plus a live investigation with real HTTP calls, plus this chaos test) records the overall result as **PASS**, and the mission's own release-blocker checklist explicitly named "a Safe Browsing failure read as SAFE" as one of a short list of unconditional release blockers — confirmed, live, not to have occurred.

[MISSING FIGURE: 41-safe-browsing-chaos-test-unknown.png]

The practical takeaway for anyone relying on either of these two providers: if you ever see `unknown` from Google Safe Browsing or urlscan.io on an investigation, treat it exactly as what it is — the check did not complete, for a real, specific, logged reason (check the provider card's own error message and the Provider Health page) — and never as a quiet, safe "nothing to worry about."